

DECLARATION

This is to certify that the work reported in NODE JS/ REACT JS/ DJANGO (CS454PC) titled **“Result Analysis System”** is a record of work done by us in the Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology, Hyderabad. No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by us and not copied from any other source.

SRINIVAS KONALA
(24261A05R0)

TIMOTHY SAXENA
(24261A05R4)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hard work and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned my efforts with success.

We would like to express our sincere thanks to, **Prof G. Chandra Mohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to **Dr. C R K Reddy, Professor and HoD, Department of CSE, MGIT**, for all the timely support and valuable suggestions during the period of project.

We are extremely thankful to **Ms. Anitha Talari, Assistant Professor, Department of CSE, MGIT**, NODE JS/ REACT JS/ DJANGO (CS454PC) faculty for her encouragement and support throughout the project.

We are also extremely thankful and indebted to our mentor, **Dr. B. Madhava Rao, Assistant Professor, Department of CSE, MGIT** for his constant guidance, encouragement, and moral support throughout the project.

Finally, we would also like to thank all the faculty and staff of the CSE Department who helped us directly or indirectly in completing this project.

SRINIVAS KONALA
(24261A05R0)

TIMOTHY SAXENA
(24261A05R4)

TABLE OF CONTENTS

CERTIFICATE	1
DECLARATION	2
ACKNOWLEDGEMENT	3
TABLE OF CONTENTS	4
LIST OF FIGURES	6
LIST OF TABLES	7
ABSTRACT	8
1. INTRODUCTION	10
2. PROBLEM STATEMENT	11
2.1 Challenges Faced by Students	11
2.2 Challenges Faced by Administration	11
2.3 The Gap This System Fills	11
3. TECHNOLOGIES USED	12
4. SOLUTION	15
4.1 System Architecture	15
4.2 Database Design	15
4.3 Excel-to-MySQL Data Pipeline	16
4.4 Excel-to-MySQL Data Pipeline	17
4.5 Role-Based Access Control	18
4.6 GPA Calculation	19
4.7 Linear Regression SGPA Projection	20
4.8 Student Dashboard — Five-Tab Design	20
4.9 Admin Dashboard Design	21
4.10 Git Workflow and Deployment	21

5. CODE	22
5.1 Import Script — Core Loop	22
5.2 JWT Authentication — Student Login	22
5.3 GPA Calculation Utility	23
5.4 Linear Regression Projection	23
5.5 Auth Middleware	24
5.6 Batch Rank Endpoint	24
6. SCREENSHOTS	25
6.1 Login Page	25
6.2 Student Dashboard Tab	25
6.3 Student Results Tab	26
6.4 Student GPA Overview Tab — Actual Trend	26
6.5 Student GPA Overview Tab — Projection	27
6.6 Student Failed Subjects Tab	28
6.7 Student Marksheet Tab and PDF Output	28
6.8 Admin Dashboard Overview	29
6.9 Admin Toppers / Ranklist	30
6.10 Admin Subject Analysis	31
6.11 Admin CGPA Distribution	31
7. FUTURE ENHANCEMENTS	32
7.1 Student Name Integration	32
7.2 Password Change Feature	32
7.3 Projection Model Improvement	32
7.4 Email Notifications	32
7.5 Mobile Application	32
7.6 Attendance Integration	32
8. CONCLUSION	33

LIST OF FIGURES

FIGURE	PAGE NO
Figure 4.1: Three-Tier Architecture Diagram	22
Figure 4.2: Entity-Relationship Diagram	23
Figure 4.3: Import Pipeline Flow	25
Figure 4.4: JWT Authentication Flow	26
Figure 4.5: Linear Regression SGPA Projection Chart	27
Figure 6.1: Login Page — Role tab switcher with Student and Admin tabs	32
Figure 6.2: Student Dashboard Tab — Stat cards, batch rank card,	33
Figure 6.3: Student Results Tab — Semester-grouped table	34
Figure 6.4: GPA Overview Tab — Cumulative CGPA trend	35
Figure 6.5: GPA Overview Tab — Linear regression projection	36
Figure 6.6: Failed Subjects Tab — Backlog list with per-semester filter	37
Figure 6.7: Marksheet Tab — Per-semester download cards	37
Figure 6.7.1: Generated PDF Marksheet	37
Figure 6.8: Admin Dashboard — Department stat cards, section breakdown	37
Figure 6.9: Admin Toppers — CGPA-ranked student list with section	37
Figure 6.10: Admin Subject Analysis — Per-subject statistics table	37
Figure 6.11: Admin CGPA Distribution — Histogram with 0.5-step buckets	37

LIST OF TABLES

TABLE	PAGE NO
Table 3.1: Technologies and their roles in the project	17
Table 4.1: Database schema overview	23
Table 4.2: Backend API route reference	23

ABSTRACT

The Result Analysis System is a full-stack web application developed for the Department of Computer Science and Engineering at Mahatma Gandhi Institute of Technology. The system addresses the absence of a centralised platform through which students can view and analyse their academic results, and administrators can monitor performance across the department.

The application supports two user roles. Students are provided with a five-tab personal dashboard covering a summary overview, subject-wise results with search and filter, a GPA overview with a cumulative CGPA trend chart and a linear regression-based SGPA projection for Year 3 and Year 4 students, a dedicated failed subjects tab, and a marksheet download facility. The Admin role provides a department-wide analytics platform covering section-wise performance, semester-wise trends, topper rankings with CGPA and SGPA-based filtering, subject-level analysis, and a CGPA distribution histogram.

The system was built using Node.js and Express.js for the backend, MySQL as the relational database, and React with Vite for the frontend. Authentication is implemented with JSON Web Tokens and bcrypt password hashing. Role-based access control is enforced at the middleware layer. The dataset comprises results for approximately 1,090 students across 35,582 result rows spanning multiple semesters, imported from departmental Excel files using a custom Node.js pipeline.

The application is fully deployed in the cloud: the React frontend on Vercel, the Express backend on Render, and the MySQL database on Railway. It is accessible through a live public URL at <https://result-analysis-three.vercel.app> and requires no local installation.

Keywords: *Result Analysis, Academic Performance, Node.js, Express.js, MySQL, React, JWT, Role-Based Access Control, Linear Regression, GPA Projection, Cloud Deployment*

1. INTRODUCTION

The Result Analysis System is a multi-role academic performance management platform designed for the CSE Department at MGIT. It replaces the traditional approach of distributing printed marksheets and maintaining results in disconnected Excel spreadsheets with a centralised web application that provides instant, authenticated access to results, automated GPA calculations, performance projections, and department-wide analytics.

The application was developed as a team project by two IV Semester students as part of the CS454PC course on Node JS / React JS / Django. The project covers a realistic academic dataset of over 35,000 result entries across five sections (CSE-1 through CSE-5) and multiple semesters, representative of actual department-scale data management requirements.

The system is structured around two user roles. The Student role provides a rich personal academic dashboard with five dedicated tabs: a summary overview with batch rank, a results browser, a GPA overview with trend analysis and future projections, a failed subjects tracker, and a marksheet generator. The Admin role provides a comprehensive department-wide analytics platform enabling data-driven oversight of academic performance.

Beyond functional features, the project demonstrates core engineering concepts from the CS454PC curriculum: RESTful API design with Express.js, relational database modelling with MySQL, client-side rendering with React, stateless JWT authentication, statistical projection using linear regression, and full-stack deployment across cloud platforms.

2. PROBLEM STATEMENT

Academic result management in most engineering departments continues to rely on printed marksheets, individually maintained Excel files, and periodic notice board displays. This approach has significant shortcomings that affect both students and the department administration.

2.1 Challenges Faced by Students

Students have no on-demand access to their complete academic record. Marksheets are distributed physically and are easily misplaced. There is no automated calculation of SGPA or CGPA, and students who need to estimate their performance in future semesters have no data-driven tool to do so. Failed subjects are not tracked centrally, and there is no single view of academic performance trends across semesters.

2.2 Challenges Faced by Administration

The department has no consolidated view of academic performance. Comparing pass percentages across sections, identifying subjects with consistently high failure rates, generating a department-wide topper list, or producing a CGPA distribution of the batch requires manually aggregating data from multiple sources. This makes academic monitoring slow and reactive rather than proactive.

2.3 The Gap This System Fills

The Result Analysis System addresses all of the above by providing a single authenticated web application that centralises result data, automates GPA computation, enables future semester performance projection using linear regression, and makes real-time analytics accessible to the admin without any spreadsheet work. The system converts a fragmented manual process into a coherent, data-driven workflow accessible from any device through a web browser.

3. TECHNOLOGIES USED

3.1 Node.js

Node.js is a JavaScript runtime environment built on Google Chrome's V8 engine. It executes JavaScript on the server side using a non-blocking, event-driven I/O model suited for handling concurrent API requests efficiently. In this project, Node.js serves as the runtime for the entire backend application.

3.2 Express.js

Express.js is a minimal web application framework for Node.js. It provides an API for defining HTTP routes, applying middleware, and handling requests and responses. The backend is structured as an Express application with route files for authentication, student operations, and admin analytics. Middleware handles CORS, JSON body parsing, and JWT-based route protection.

3.3 MySQL

MySQL is an open-source relational database management system. The project uses four tables: students, results, faculty, and admins. The results table contains over 35,000 rows. MySQL's support for JOIN queries, aggregate functions, and GROUP BY operations is central to GPA calculations and analytics. The database is hosted on Railway and connected via a connection pool using the mysql2 library.

3.4 React and Vite

React is a JavaScript library for building user interfaces through reusable components and a virtual DOM. Vite is the build tool and development server used to bundle the React application. The frontend uses a custom client-side router implemented directly in App.jsx using window.location.pathname, keeping the dependency footprint minimal. All five student dashboard tabs and both admin dashboard pages are React components.

3.5 JSON Web Tokens (JWT)

JSON Web Tokens are used for stateless authentication. On successful login, the server signs a token containing the user's ID and role. This token is stored in the browser's localStorage and attached to every API request as a Bearer token in the Authorization header. The server verifies the signature on each request without a database lookup, making authentication scalable and sessionless.

3.9 xlsx

The xlsx library reads Excel files in Node.js. It is used exclusively in the import script to parse departmental result files, converting each worksheet row into a JavaScript object for database insertion.

3.10 Deployment Platforms

Three cloud platforms handle deployment. Vercel hosts the React frontend, building and deploying it automatically on every push to the dev branch. Render hosts the Node.js backend as a persistent web service with environment variables for database credentials and the JWT secret. Railway hosts the MySQL database and provides a publicly accessible connection endpoint. All three services are connected to the same GitHub repository and deploy automatically when the dev branch is updated.

4.8 Student Dashboard — Five-Tab Design

The student interface is organised into five tabs to avoid information overload while keeping all features accessible from a single page.

Tab 1: Dashboard

The Dashboard tab serves as the landing screen after login. It displays the MGIT logo, a welcome message with the student's HT number, and four summary stat cards: overall CGPA, latest SGPA, number of semesters completed, and backlog count. A batch rank card shows the student's rank within their batch computed by the `/api/student/rank` endpoint. The most recent semester's results are displayed in a summary table below the cards.

Tab 2: Results

The Results tab displays all subject results grouped by semester. A semester dropdown filter and a text search box allow the student to quickly find specific subjects or narrow down to a particular term. Each semester group shows the SGPA for that term. Failed subjects are visually highlighted with a red indicator and a faint red row background.

Tab 3: GPA Overview

The GPA Overview tab contains two charts. The first is a cumulative CGPA trend line showing how the student's overall CGPA has evolved as each semester was completed. The second chart shows the SGPA trend with the linear regression projection for Year 3 and Year 4 students. Both charts are rendered as pure SVG with no external charting dependencies. A table below the charts lists SGPA per semester alongside grade statistics.

Tab 4: Failed Subjects

The Failed Subjects tab provides a dedicated view of all subjects in which the student has received an F grade. A semester filter allows the student to check whether failed subjects belong to a specific term. Each entry shows the subject code, subject name, semester, marks obtained, and the grade. This tab is intentionally separated from the main results view to make backlogs immediately visible without scrolling through a full results table.

Tab 5: Marksheet

The Marksheet tab displays one download card per completed semester. Each card shows the semester number and the SGPA for that term. Clicking the download button triggers a request to the backend, which generates an A4 PDF marksheet using `pdfkit` and streams it to the browser. The PDF includes the college header, student details, a subject table, summary boxes, and a colour-coded pass/fail banner.

4.9 Admin Dashboard Design

The admin interface is a single-page application with a persistent left sidebar for navigation. Four tabs are available: Department Overview, Toppers / Ranklist, Subject Analysis, and CGPA Distribution. Sidebar filters for semester, section, and year apply to whichever tab is currently active. A CSV export button triggers a full data download with the active filters applied.

4.10 Git Workflow and Deployment

Development followed a structured Git branching strategy. The main branch holds stable, tested code. The dev branch is the integration branch and is the branch from which both Vercel and Render deploy automatically on push. Feature work is done on short-lived feature branches created off dev, which are merged back via pull requests after review by the second team member.

The deployment pipeline requires no manual steps after a push. Vercel detects the push to dev, runs `npm run build` to produce the static React bundle, and serves it from a global CDN with a custom SPA rewrite rule in `vercel.json`. Render detects the push, runs `npm install`, and restarts the Express server. Railway requires no redeployment as the database persists independently of application deployments.

5. CODE

This chapter presents key code excerpts from the project. The full source code is available at <https://github.com/timothy-saxena/result-analysis> on the dev branch.

5.1 Import Script — Core Loop (backend/scripts/import.js)

The import script processes one Excel file per run. For each row it creates a student account if one does not exist and upserts the result record using ON DUPLICATE KEY UPDATE, making repeated runs safe.

```
for (const row of rows) {
  const ht_no = String(row["HT_NO"]).trim();

  // Create student if not already present
  const [existing] = await db.query(
    "SELECT ht_no FROM students WHERE ht_no = ?", [ht_no]
  );
  if (existing.length === 0) {
    const hash = await bcrypt.hash(`${ht_no}@123`, 10);
    const meta = inferStudentMeta(ht_no);
    await db.query(
      "INSERT INTO students VALUES (?, ?, ?, ?, ?)",
      [ht_no, hash, meta.branch, meta.year, meta.section]
    );
  }

  // Upsert result - safe to re-run on the same file
  await db.query(
    `INSERT INTO results
      (ht_no, semester, course_code, course_name,
       cie_marks, see_marks, total_marks,
       grade_letter, grade_points, credits)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
      ON DUPLICATE KEY UPDATE
        total_marks = VALUES(total_marks),
        grade_letter = VALUES(grade_letter),
        grade_points = VALUES(grade_points)`,
    [ht_no, semester, courseCode, /* ... */]
  );
}
```

5.2 JWT Authentication — Student Login (backend/routes/auth.js)

The login route queries the student record, compares the submitted password against the stored bcrypt hash, and returns a signed JWT on success.

```
router.post("/student-login", async (req, res) => {
  const { ht_no, password } = req.body;
  const [rows] = await db.query(
    "SELECT * FROM students WHERE ht_no = ?", [ht_no]
  );
  if (rows.length === 0)
    return res.status(401).json({ error: "Invalid credentials." });

  const valid = await bcrypt.compare(password, rows[0].password_hash);
```

```

if (!valid)
  return res.status(401).json({ error: "Invalid credentials." });

const token = jwt.sign(
  { id: ht_no, role: "student" },
  process.env.JWT_SECRET,
  { expiresIn: "7d" }
);
res.json({ token, role: "student", ht_no });
});

```

5.3 GPA Calculation Utility (backend/utils/gpa.js)

The SGPA function is the shared source of truth used by both student routes and admin analytics.

```

const calculateSGPA = (results) => {
  let totalWeighted = 0;
  let totalCredits = 0;
  for (const row of results) {
    if (row.credits > 0) {
      totalWeighted += row.grade_points * row.credits;
      totalCredits += row.credits;
    }
  }
  if (totalCredits === 0) return 0;
  return parseFloat((totalWeighted / totalCredits).toFixed(2));
};

// CGPA = same formula applied across all semesters combined
const calculateCGPA = (allResults) => calculateSGPA(allResults);

```

5.4 Linear Regression Projection (frontend — GPA Overview tab)

The following function computes the slope and intercept of the best-fit line through the student's historical SGPA values, then projects SGPA for future semesters.

```

function linearRegression(points) {
  const n = points.length;
  const sumX = points.reduce((a, p) => a + p.x, 0);
  const sumY = points.reduce((a, p) => a + p.y, 0);
  const sumXY = points.reduce((a, p) => a + p.x * p.y, 0);
  const sumX2 = points.reduce((a, p) => a + p.x * p.x, 0);
  const m = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
  const b = (sumY - m * sumX) / n;
  return { m, b };
}

function projectSGPA(sgpaData, futureSemesters) {
  const points = sgpaData.map(s => ({ x: s.semester, y: s.sgpa }));
  const { m, b } = linearRegression(points);
  return futureSemesters.map(sem => ({
    semester: sem,
    projected: Math.min(10, Math.max(0,
      parseFloat((m * sem + b).toFixed(2))
    )),
  }));
}

```

5.5 Auth Middleware (backend/middleware/auth.js)

Every protected route passes through `verifyToken` and `requireRole` before the handler runs.

```
const verifyToken = (req, res, next) => {
  const token = req.headers["authorization"]?.split(" ")[1];
  if (!token)
    return res.status(401).json({ error: "No token provided." });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(403).json({ error: "Invalid or expired token." });
  }
};

const requireRole = (roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ error: "Access denied." });
  next();
};
```

5.6 Batch Rank Endpoint (backend/routes/student.js)

The `/student/rank` endpoint computes the student's rank within their batch by computing CGPA for every student and counting how many have a higher value.

```
router.get("/rank", async (req, res) => {
  const ht_no = req.user.id;

  // Get all students in the same batch (same year prefix)
  const batchPrefix = ht_no.substring(0, 2);
  const [allStudents] = await db.query(
    `SELECT r.ht_no,
      ROUND(SUM(r.grade_points * r.credits) /
        NULLIF(SUM(CASE WHEN r.credits > 0
          THEN r.credits ELSE 0 END), 0), 2) AS cgpa
      FROM results r
      JOIN students s ON r.ht_no = s.ht_no
      WHERE r.ht_no LIKE ? AND r.credits > 0
      GROUP BY r.ht_no`,
    [`${batchPrefix}%`]
  );

  const myCGPA = allStudents.find(s => s.ht_no === ht_no)?.cgpa || 0;
  const rank = allStudents.filter(s => s.cgpa > myCGPA).length + 1;
  const total = allStudents.length;

  res.json({ rank, total,
    percentile: parseFloat(((1 - rank / total) * 100).toFixed(1)),
    cgpa: myCGPA
  });
});
```

7. ENHANCEMENTS

7.1 Student Name Integration

The current dataset contains no student names — students are identified solely by their hall ticket number. The database schema includes a name column in the students table that is currently unpopulated. Once the department provides a name mapping, the import script can be extended to populate this field and all student-facing screens can display a personalised greeting alongside the HT number.

7.2 Password Change Feature

Students currently use a default password that cannot be changed through the application. Adding a password change route and a form on the dashboard would allow students to set a personal password, improving both usability and security.

7.3 Projection Model Improvement

The current SGPA projection uses simple linear regression, which assumes a linear trend. A polynomial regression or weighted regression model (where more recent semesters carry greater weight) would produce more accurate projections for students whose performance has been non-linear. This would be a natural extension once more semester data is available.

7.4 Email Notifications

Automated email notifications could alert students when marks are updated by the admin or when a new semester's results are imported. This would eliminate the need for students to check the portal manually and would increase platform engagement.

7.5 Mobile Application

A React Native or Flutter mobile application would provide a more native experience for the student role. Push notifications for result updates and direct camera access for document scanning would become possible through a mobile client.

7.6 Attendance Integration

Integrating attendance data alongside result data would allow the admin dashboard to correlate attendance percentages with academic performance, providing richer insights into the factors that affect student outcomes across sections and subjects.

8. CONCLUSION

The Result Analysis System successfully demonstrates the practical application of Node.js, Express.js, MySQL, and React in building a production-grade, multi-role web application. Developed as a two-person team project, it is fully deployed and publicly accessible, making it a functional deliverable beyond a purely academic exercise.

The system manages result data for 1,090 students across 35,582 result entries, supports two distinct user roles with JWT-based role-specific access control, automates SGPA and CGPA calculation, implements a linear regression-based SGPA projection for advanced students, generates PDF marksheets, enables batch rank computation, and provides comprehensive department-wide analytics across sections and semesters.

The student dashboard's five-tab design — covering an overview with batch rank, a searchable results browser, a GPA overview with projection charts, a failed subjects tracker, and marksheet downloads — demonstrates how a complex feature set can be organised for clarity and usability. The linear regression projection in particular represents a meaningful application of statistical reasoning to a practical academic problem.

Key engineering concepts applied include RESTful API design, stateless JWT authentication, relational database schema design with foreign key constraints and unique indexes, bcrypt password security, middleware-based route protection, cloud deployment across multiple platforms, and collaborative Git-based development. The project encountered and resolved real-world challenges including MySQL strict mode compatibility, CORS configuration for cross-origin deployment, PDF generation library incompatibility with serverless hosting, and SPA routing on static CDN hosting.

The codebase is designed for extensibility. The modular route structure, shared utility functions, and clean separation between the frontend and backend make it straightforward to add the faculty role, additional analytics, or extended reporting without disrupting existing functionality.